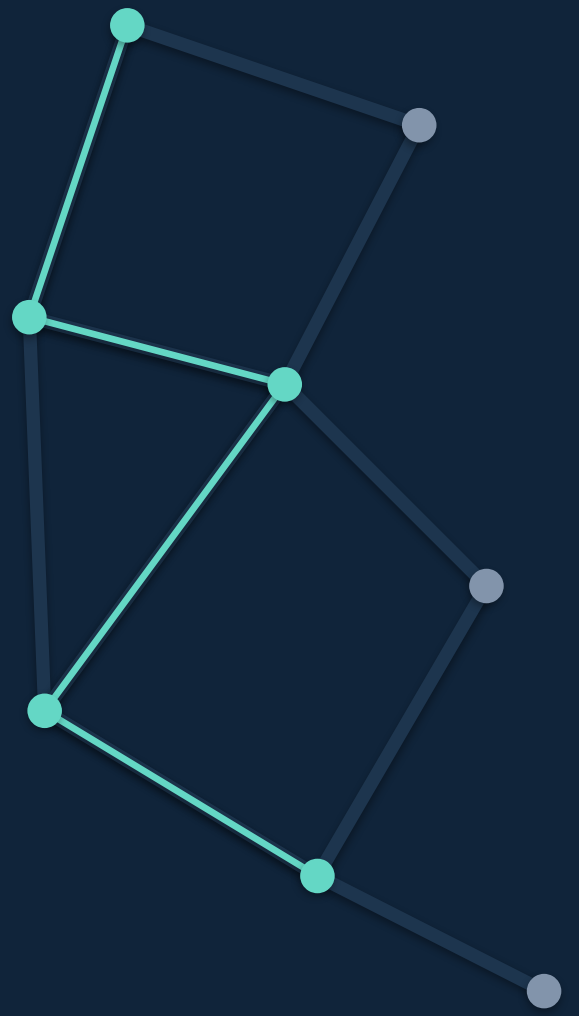
Decoupled Intelligence

A Multi-Agent LLM Framework for Controllable Traffic Scenario Generation in SUMO

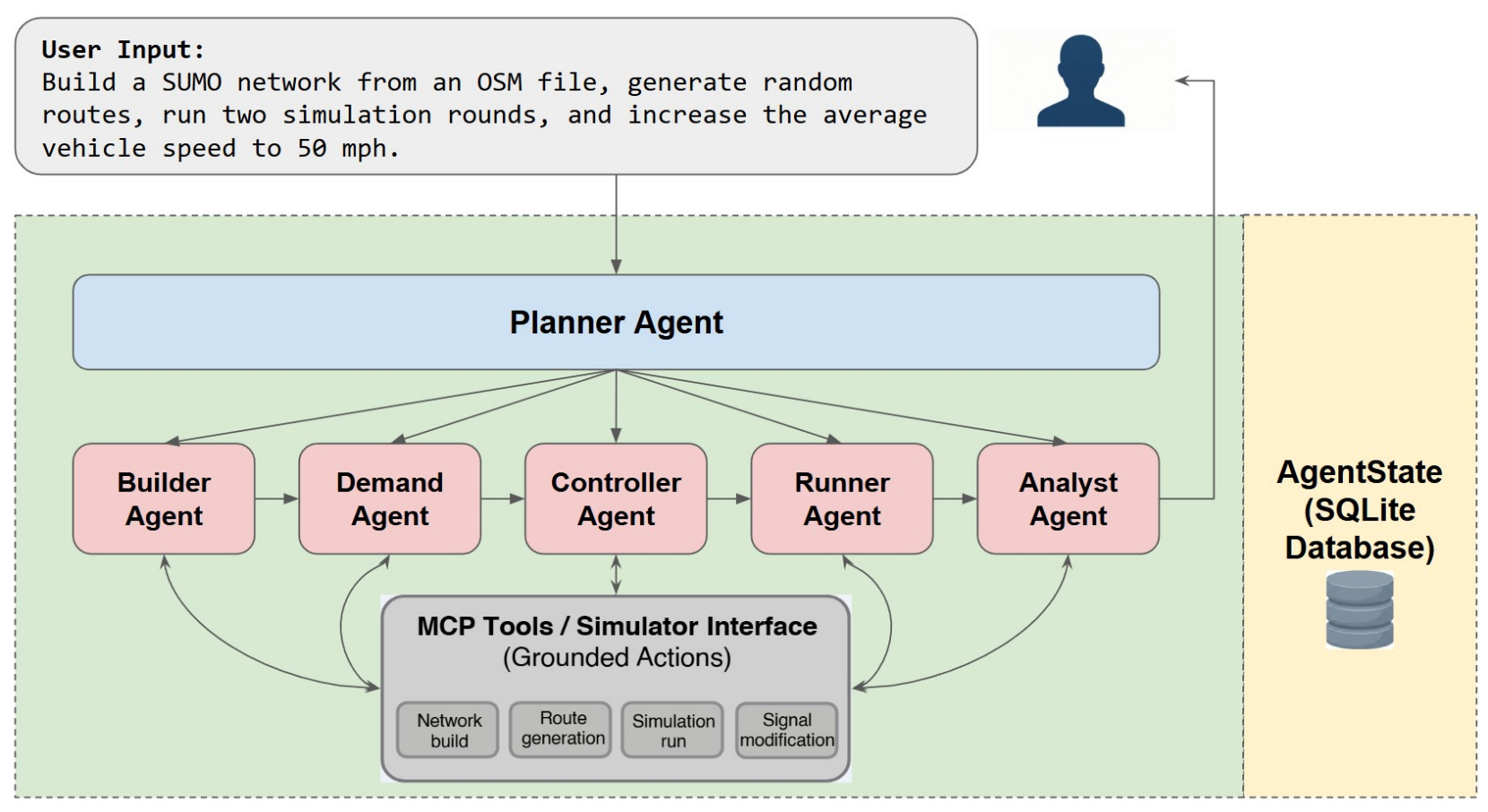


INTENT → EXECUTION

Shuyang Li · Ruimin Ke

Civil and Environmental Engineering
Rensselaer Polytechnic Institute

One traffic question spans an entire workflow



01

Understand intent

A natural-language traffic objective

02

Coordinate tools

Network, demand, execution, analysis

03

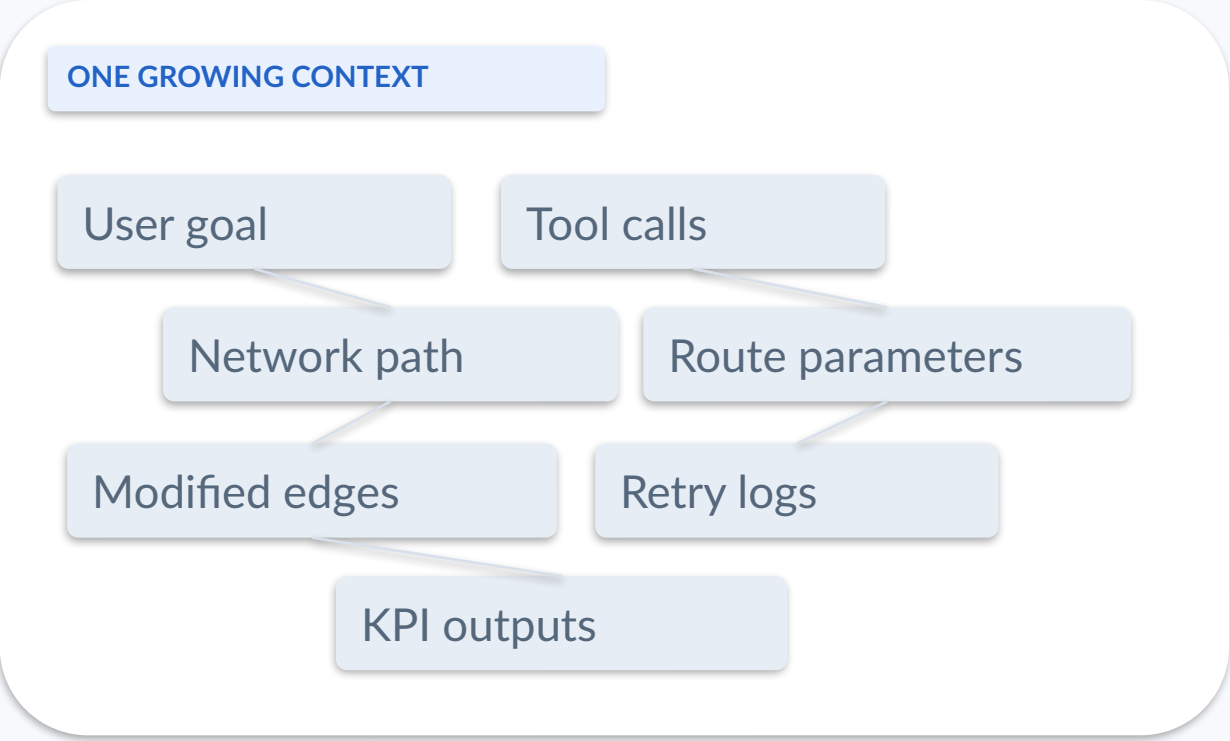
Preserve state

Carry valid artifacts across dependent steps

Source: original paper Fig. 1 (reproduced)

A single context must remember every dependency

The paper's baseline is a monolithic ReAct agent with all tools and execution history.



- **Path reference errors**

  A downstream step points to the wrong file.

- **Parameter inconsistency**

  Physical values drift across tool calls.

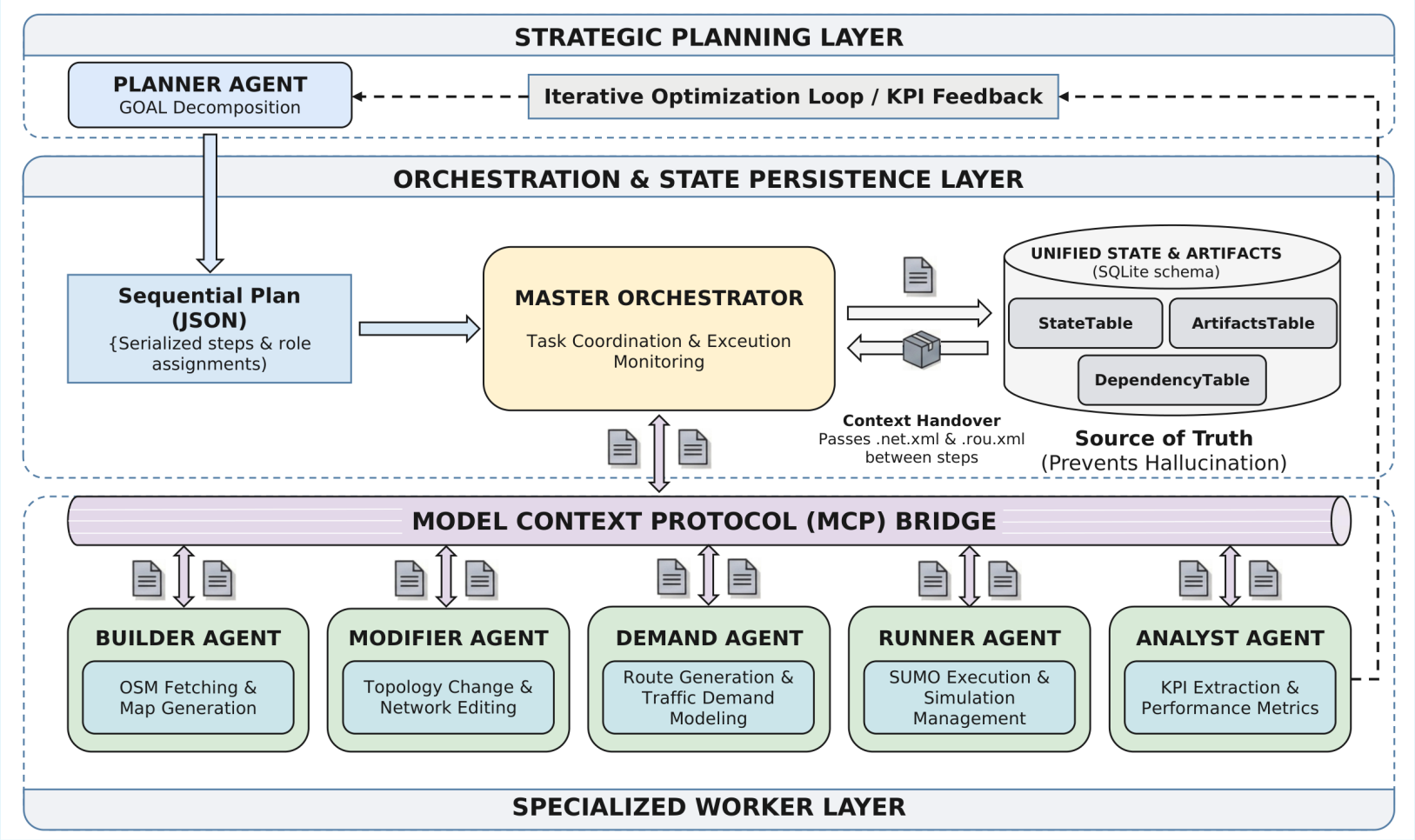
- **Reasoning drift**

  Retries lose track of the original objective.

Design question: which responsibilities should be explicit in the system?

Source: Sec. IV-A, IV-C, IV-E

Separate reasoning roles. Make state explicit.



01

Planner

Turn intent into an ordered plan.

02

Orchestrator

Track artifacts and dependencies.

03

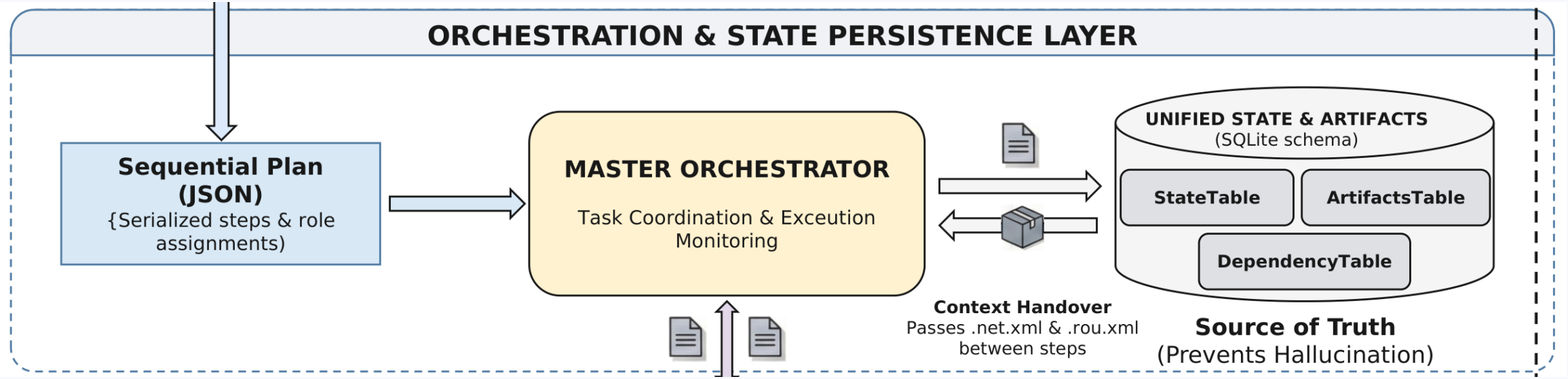
Workers

Execute scoped tasks through MCP tools.

Source: original paper Fig. 2 (reproduced)

Route generation uses the validated network

A closer look at the Orchestrator and persistent artifact repository.



Builder
Original network

→

Modifier
Closure applied

→

Demand
Routes on updated network

→

Runner
Matching network + routes

Dependency rule: apply topology changes before generating demand.

Source: Fig. 2, enlarged state layer; Sec. III-C

The Planner turns intent into ordered contracts

Original user input

1 User Input

Create a 1.5 km simulation in Boston. Close 'Boylston Street', send 800 veh/hour from 'Newbury Street' to 'Beacon Street', execute the run, and report the average waiting time.

Ordered plan

Builder → Modifier → Demand → Runner → Analyst

One active role per step; explicit parameters.

Network shown in Fig. 3

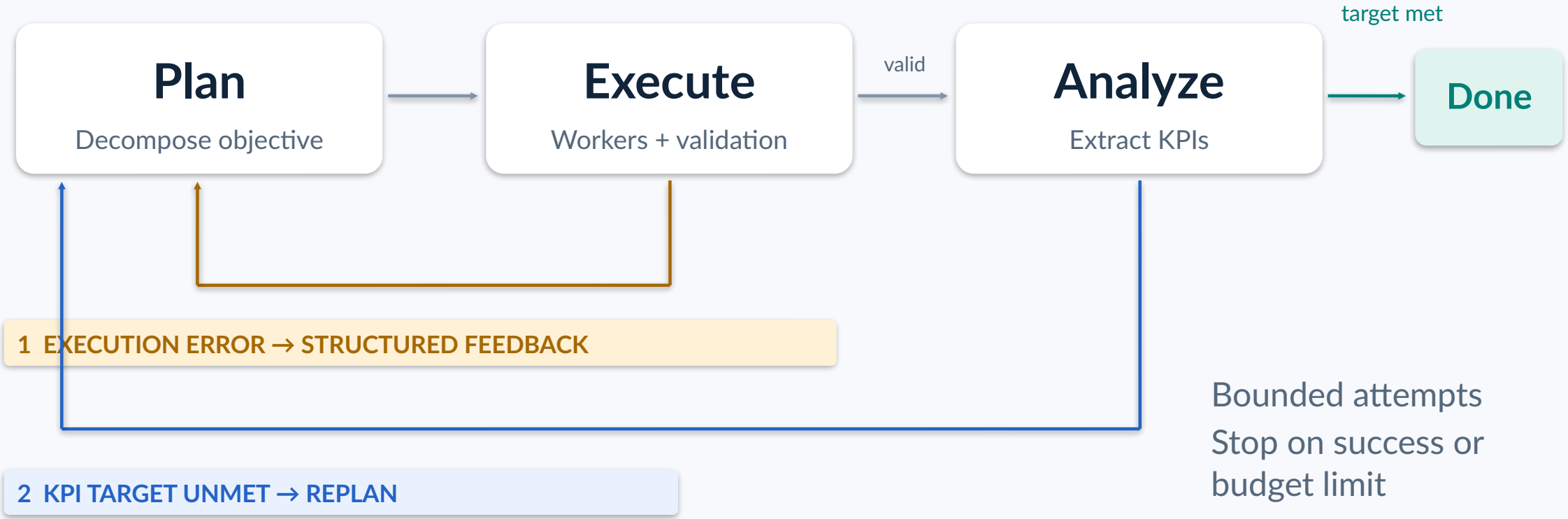


Original paper screenshot

Source: original Fig. 3 details; full figure in backup B5

Two feedback loops address two different problems

Execution repair handles tool failures; KPI feedback revisits the scenario objective.



Bounded attempts

Stop on success or budget limit

Source: Sec. III-B; Algorithm 1

30 tasks test increasing dependency complexity

SUMO 1.21.0 · Qwen3-30B-Coder · monolithic ReAct vs. the proposed framework

L1

Basic

Build a geographic network + qualitative traffic

L2

Intermediate

Specify origin–destination flow + align tool parameters

L3

Complex

Modify network topology before demand generation

WHAT IS MEASURED?

Success rate ↑

Valid configuration + error-free execution

Average tokens ↓

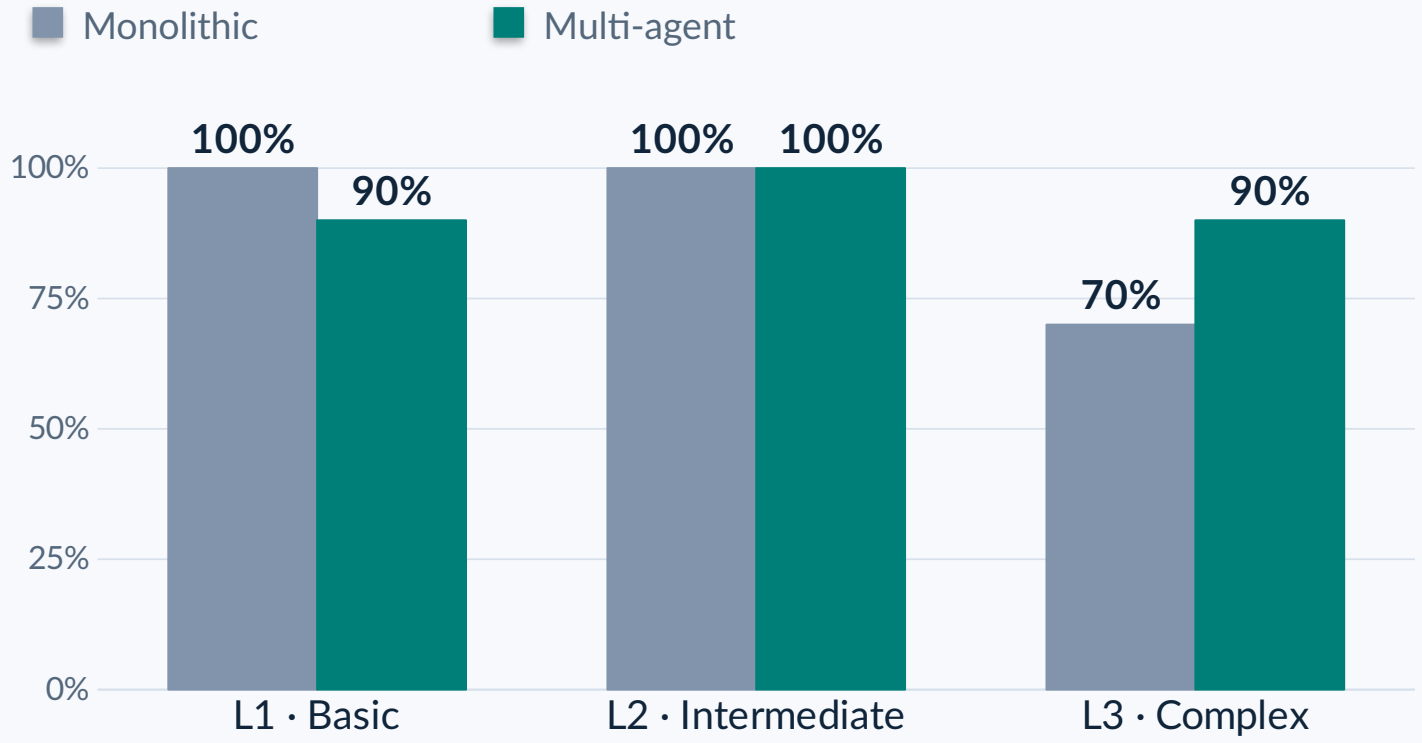
Total LLM input + output tokens

Time-to-Insight ↓

User request → first valid KPI

Complex-task success rises from 70% to 90%

Success rate by task complexity (%) ↑



L3 / COMPLEX

+20

percentage points

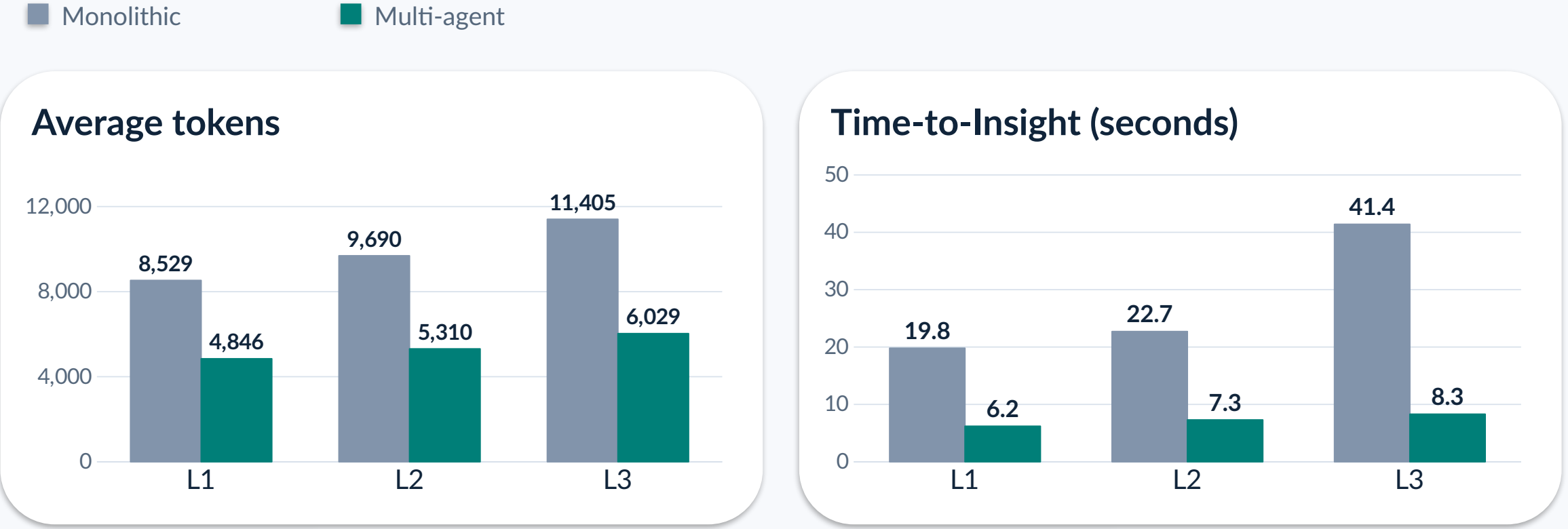
The gain is concentrated in harder workflows.

L1 is 10 points lower; L2 is unchanged. Role separation helps most when dependencies grow.

Source: Table I; Sec. IV-C

Fewer tokens and a shorter path to the first KPI

Reported averages in the architecture comparison; lower is better.



L3: 47.1% fewer tokens

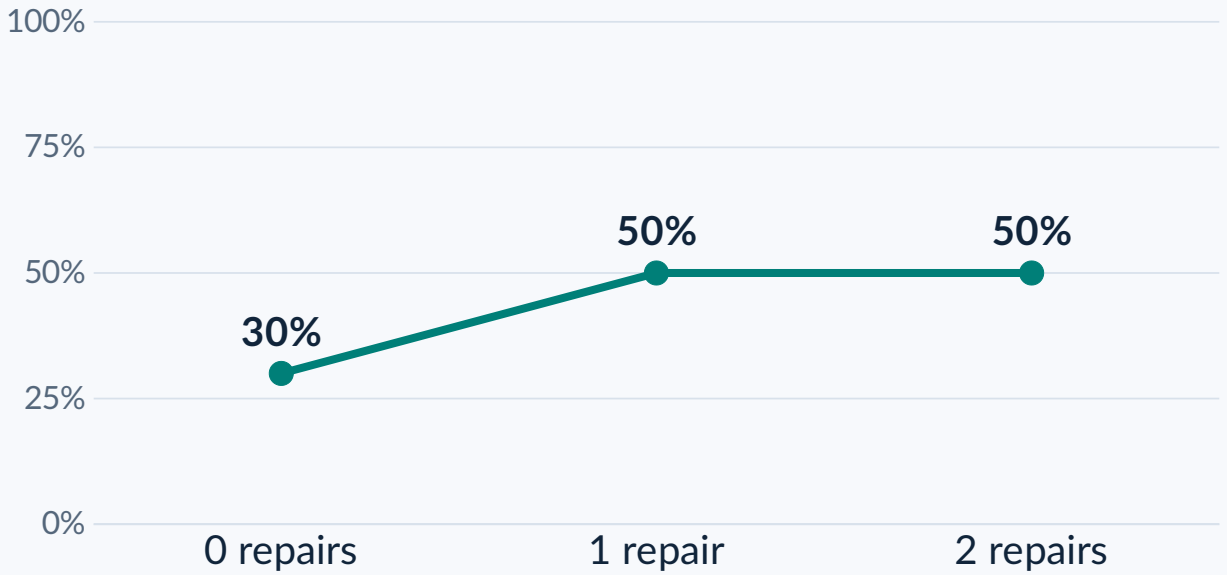
L3: 41.4 s → 8.3 s / ≈5.0× speedup

Source: Table I; percentages and ratios calculated from reported values

One repair attempt helps; a second adds cost

Separate experiment: compare 0, 1, and 2 maximum repair attempts within Table II.

L2 and L3 success rate (%)



L2 and L3 have identical reported success rates.

L3 average tokens

0 repairs	3,981.6
1 repair	6,609.9
2 repairs	9,214.1

More retries ≠ higher success

Simple L1 tasks: 90% → 80% → 90%; unnecessary repair can misread benign warnings.

Source: Table II; Sec. IV-D

What this evidence establishes—and what remains open

A promising workflow result with a bounded empirical scope.

Supported by this study

Higher L3 execution success

Lower tokens and reported TTI

Recovery gains with one repair

Open questions

30 tasks; one evaluated backbone

Components not isolated individually

KPI optimization lacks a quantitative convergence or optimality result

Next: broader benchmarks, component-level ablations, and measured KPI optimization.

Source: Sec. IV–V; reporting limitations identified from the paper

TAKEAWAY

Reliable automation needs explicit coordination

Specialized reasoning + persistent artifacts + bounded feedback

90%

L3 task success

70% monolithic baseline

−47%

L3 token use

11,404.7 → 6,028.7

≈5×

L3 TTI speedup

41.4 s → 8.3 s

Natural-language objectives → executable, inspectable SUMO workflows.

Evidence: Table I, complex tasks (L3)

Thank you

Questions?

Decoupled Intelligence

Shuyang Li · Ruimin Ke
Rensselaer Polytechnic Institute

Supported by the IBM–RPI Future of Computing Research Collaboration.

Full results: monolithic vs. multi-agent

Exact values from Table I. ↑ Higher is better; ↓ lower is better.

Complexity	Metric	Monolithic	Multi-agent
L1 · Basic	Success rate (%) ↑	100.0	90.0
	Average tokens ↓	8,529.2	4,845.5
	TTI (s) ↓	19.8	6.2
L2 · Intermediate	Success rate (%) ↑	100.0	100.0
	Average tokens ↓	9,690.3	5,310.1
	TTI (s) ↓	22.7	7.3
L3 · Complex	Success rate (%) ↑	70.0	90.0
	Average tokens ↓	11,404.7	6,028.7
	TTI (s) ↓	41.4	8.3

Overall architecture comparison; individual components are not isolated in this table.

Full results: the repair-budget experiment

Exact values from Table II. Compare repair budgets within this experiment.

Complexity	Metric	0 repairs	1 repair	2 repairs
L1 · Basic	Success rate (%) ↑	90.0	80.0	90.0
	Average tokens ↓	4,969.1	5,530.3	6,752.6
L2 · Intermediate	Success rate (%) ↑	30.0	50.0	50.0
	Average tokens ↓	3,601.6	6,829.5	8,484.8
L3 · Complex	Success rate (%) ↑	30.0	50.0	50.0
	Average tokens ↓	3,981.6	6,609.9	9,214.1

No additional L2/L3 success gain from a second attempt; token use continues to rise.

Source: Table II; Sec. IV-D

What each worker receives and produces

Role contracts and artifact dependencies, simplified from the methodology and prompt appendices.

Worker	Required context	Responsibility	Output
Builder	City / geographic extent	Build network	.net.xml
Modifier	Network + requested edit	Apply topology / TLS edit	Updated .net.xml
Demand	Validated network + flow / O–D	Generate matching demand	.rou.xml
Runner	Network + routes + run settings	Configure and execute	.sumocfg + outputs
Analyst	Simulation outputs + metric	Aggregate KPIs	Metrics + feedback

One active role per planned step

Skip Modifier when no edit is requested

Source: Sec. III-C; Appendix II–III

Answers to likely design questions

Why not a stronger single agent?	This study compares one backbone and one monolithic ReAct design. Broader baselines remain to be tested.
Does MCP itself prevent path errors?	MCP standardizes tool access. The Orchestrator manages validated artifact references and dependencies.
Is traffic KPI optimization proven?	The feedback mechanism is designed for it. Reported quantitative results focus on execution and repair.
Do more agents or retries always help?	No. L1 success is lower in Table I; a second repair adds tokens without improving L2/L3 success in Table II.

Original Planner workflow from the paper

1 User Input

Create a 1.5 km simulation in Boston. Close 'Boylston Street', send 800 veh/hour from 'Newbury Street' to 'Beacon Street', execute the run, and report the average waiting time.

2 Planner Agent Prompt

```
Planner agent:
    You are a TOP-LEVEL planner for a SUMO traffic
simulation multi-agent system.
    Your job: Read USER_INPUT and the current simulation
state. Decompose the task into a sequence of steps (1-5).
For EACH step, write short natural-language instructions
that include NECESSARY physical parameters for downstream
agents.

    Parameter Alignment Contract (Include these in your
natural language instructions):
    1. For Builder: Must specify "city_name",
"distance_miles" (e.g. 1.5), and "volume" (total trips).
    Other contracts for role agents ......

    Instruction Style Examples:
    planner examples
```

3 Planner Agent Output

```
'plan':
[{'step_id': 1, 'goal': 'Build a 1.5 km
simulation network in Boston',
'agent_prompts': {'builder': 'Build a
network for Boston with 1.5 miles
radius and 1000 volume.', 'demand': '',
'modifier': '', 'runner_planner': '',
'analyst': ''}},

{'step_id': 2, 'goal': 'Close Boylston
Street in the network',
'agent_prompts': {'builder': '',
'demand': '', 'modifier': "Remove the
edge with ID 'Boylston Street' from the
generated network.", 'runner_planner':
'', 'analyst': ''}},

......

{'step_id': 5, 'goal': 'Analyze the
simulation results for average waiting
time', 'agent_prompts': {'builder': '',
'demand': '', 'modifier': '',
'runner_planner': '', 'analyst':
'Analyze the simulation results
focusing on waiting_time.'}}]
```

4 Network generation

```
Builder Agent:
Input:
Build a network for Boston with 1.5
miles radius and 1000 volume.
Output:
{'action': 'BUILD_NET', 'params':
{'type': 'build_from_realworld',
'city_name': 'Boston',
'distance_miles': 0.932056788,
'volume': 1000}}
```



Source: paper Fig. 3, reproduced without changes